



Date: __ / __ / ____

Experiment 1

1. a. Write a C program function to perform a **linear search** on a list of integers. The function should take two parameters - the array of integers and the target value to be searched. If the target value is present in the list, the function should return all the index of the occurrence of the target value. If the target value is not found, print Not found message.

Constraints

1. Must use a function called linearSearch
2. Must pass an array to the linearSearch function
3. The function must return set of all matching indices back to the main function.
4. Print the indices in main function.

Program

```
#include <stdio.h>
```

```
// Function to perform linear search
```

```
void linearSearch(int arr[], int size, int target, int result[], int *resultSize) {
```

```
    *resultSize = 0; // Initialize the resultSize to 0
```

```
    // Iterate through the array
```

```
    for (int i = 0; i < size; i++) {
```

```
        // Check if the current element matches the target
```

```
        if (arr[i] == target) {
```

```
            // Add the matching index to the result array
```

```
            result[*resultSize] = i;
```

```
            (*resultSize)++; // Increment the resultSize
```

```
        }
```

```
    }
```

```
}
```

```
int main() {
```

```
    int size, target;
```

```
    // Input the size of the array
```

```
    printf("Enter the size of the array: ");
```

```
    scanf("%d", &size);
```

```
    int arr[size];
```

```
    // Input the elements of the array
```

```
    printf("Enter the elements of the array:\n");
```

```
    for (int i = 0; i < size; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```



```
// Input the target value to search
printf("Enter the target value to search: ");
scanf("%d", &target);

// Declare an array to store matching indices
int result[size];
int resultSize;

// Perform linear search
linearSearch(arr, size, target, result, &resultSize);

// Print the matching indices
if (resultSize > 0)
{
    printf("Matching indices for target %d: ", target);
    for (int i = 0; i < resultSize; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");
} else {
    printf("No matching indices found for target %d.\n", target);
}

return 0;
}
```

Sample Output

Enter the size of the array: 5
Enter the elements of the array:
1 2 3 4 5
Enter the target value to search: 6
No matching indices found for target 6.

Enter the size of the array: 5
Enter the elements of the array:
1 1 1 1 1
Enter the target value to search: 1
Matching indices for target 1: 0 1 2 3 4

Enter the size of the array: 5
Enter the elements of the array:
1 2 3 4 5
Enter the target value to search: 5
Matching indices for target 5: 4



1.b. Write a C program that includes two functions. The first function, named **validateSortedArray**, should take an array of integers as input and validate if the array is sorted in ascending order. The second function, named **binarySearch**, should perform binary search on a sorted list of integers. The **binarySearch** function should take two parameters - the validated sorted list of integers and the target value to be searched. If the target value is present in the list, the function should return the index of the target value. If the target value is not found, the function should return -1. Implement both functions and provide an example of their usage.

Program

```
#include <stdio.h>
// Function to validate if the array is sorted in ascending order
int validateSortedArray(int arr[], int size)
{
    for (int i = 0; i < size - 1; ++i)
    {
        if (arr[i] > arr[i + 1])
        {
            return 0; // Not sorted
        }
    }
    return 1; // Sorted
}

// Function to perform binary search on a sorted list of integers
int binarySearch(int arr[], int size, int target)
{
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target)
        {
            return mid; // Target value found, return the index
        }
        else if (arr[mid] < target)
        {
            low = mid + 1; // Search the right half
        }
        else
        {
            high = mid - 1; // Search the left half
        }
    }
}
```



```
        return -1; // Target value not found
    }

    int main() {
        int n, target;

        printf("Enter the number of elements: ");
        scanf("%d", &n);

        int arr[n];

        printf("Enter the sorted elements:\n");
        for (int i = 0; i < n; ++i)
        {
            scanf("%d", &arr[i]);
        }

        // Validate if the array is sorted
        if (validateSortedArray(arr, n))
        {
            printf("Enter the element to be searched: ");
            scanf("%d", &target);

            // Perform binary search
            int result = binarySearch(arr, n, target);

            if (result != -1)
            {
                printf("Target value %d found at index: %d\n", target, result);
            }
            else
            {
                printf("Target value %d not found in the array.\n", target);
            }
        }
        else
        {
            printf("The entered array is not sorted.\n");
        }

        return 0;
    }
```

Sample Output

```
Enter the number of elements: 5
Enter the sorted elements:
5 4 3 2 1
The entered array is not sorted.
```



Enter the number of elements: 5
Enter the sorted elements:
1 2 3 4 5
Enter the element to be searched: 5
Target value 5 found at index: 4

Enter the number of elements: 5
Enter the sorted elements:
1 2 3 4 5
Enter the element to be searched: 9
Target value 9 not found in the array

Viva Questions

1. Discuss the advantages and limitations of binary search compared to linear search.
2. Explain the time complexity of binary search. How does it compare to linear search?
3. In what situations might you prefer using linear search over binary search?